

Mixed-Signal Tutorial

VCS® AMS Mixed-Signal

Simulation Using SPICE Top

SYNOPSYS®

Contents

- Overview3
 - Design Files3
- Setting up the Environment for CustomSim and VCS.....4
- Running an Analog-only (CustomSim) Simulation5
 - Viewing Waveforms in Custom WaveView5
- Running a Two-Step Mixed-Signal VCS AMS Simulation.....8
- Running a Three-Step Mixed-Signal VCS AMS Simulation10
- Understanding Mixed-Signal Simulation Report Files.....11
- Summary.....14

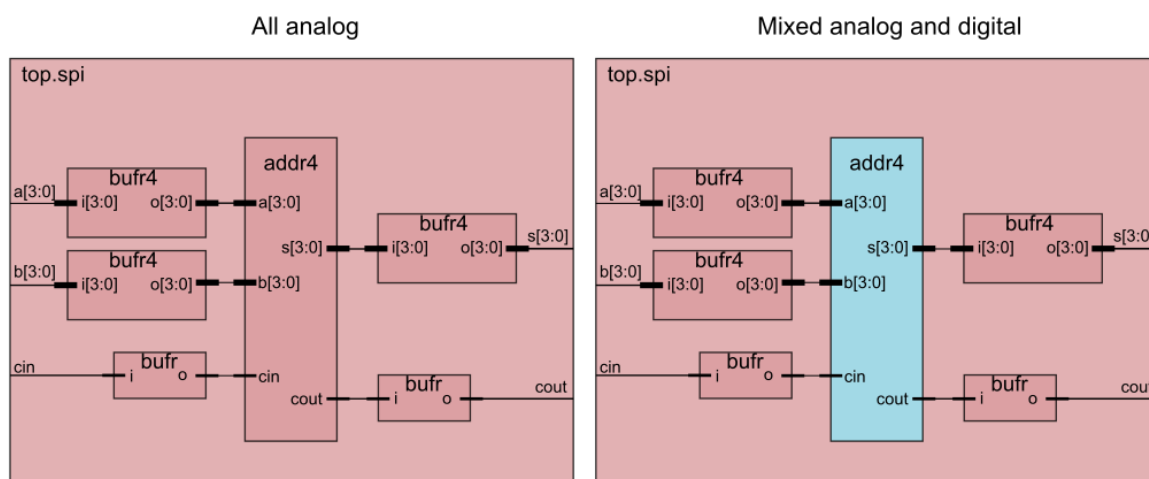
Overview

This tutorial demonstrates a simple VCS® AMS (CustomSim™ + VCS) mixed-signal simulation. The design is configured to use a top-level SPICE netlist that instantiates a Verilog cell.

Design Files

This tutorial uses a 4-bit adder circuit with three inputs (a, b, and cin) and two outputs (s and cout). 4-bit and 1-bit buffers are used to buffer the signals connecting to the 4-bit adder. The top-level SPICE netlist, top.spi, instantiates the 4-bit adder and buffers. The adder.vec file contains stimulus for the simulation. The mixed-signal simulation replaces the SPICE-level 4-bit adder with a Verilog model.

Figure 1 *Tutorial Design*



The tutorial includes three script files to run the simulation:

- runXa : An all-analog simulation run script that models the entire design in SPICE.
- runVcsAms : A mixed-signal VCS AMS simulation run script that uses a top-level SPICE netlist and replaces the 4-bit adder with a digital model.
- runVcsAms_3step : A mixed-signal VCS AMS simulation run script that uses a top-level SPICE netlist and replaces the 4-bit adder with a digital model. The simulation is compiled and run by using a three-step flow.

The following table describes the files contained in the tutorial.

File	Description
adder.vec	Simulation input and expected output signals and values

addr4.v	Verilog behavioral description of the 4-bit adder
fsdbDump.v	Verilog module used to generate FSDB output
synopsys_sim.setup	Library configuration file for mixed-HDL designs
top.spi	SPICE circuit definition for the top-level design that contains the 4-bit adder and buffer instances
vcsAD.init	Mixed-signal control file
vcsAD_3step.init	Mixed-signal control file for three-step flow
xa.cfg	CustomSim configuration file
runXa	Script for CustomSim-only compile and run
runVcsAms	Script for VCS AMS compile and run with two-step flow
runVcsAms_3step	Script for VCS AMS compile and run with three-step flow
CLEAN	A script that removes all result and intermediate directories generated during the run. The original files in the tutorial are not deleted.
README	Brief description of this tutorial

Setting up the Environment for CustomSim and VCS

To run the tutorial, you must set the path and environment variables to run the CustomSim and VCS tools. To set the variables:

1. Source the CSHRC_xa shell script file provided in the CustomSim installation directory to set the path to the CustomSim executable. Use the SHRC_xa shell script if you are using the Bourne, Korn, or Bash shells.

```
# C-shell
% source /xa_installation_directory/CSHRC_xa

# sh / bash / ksh shell
$ source /xa_installation_directory/SHRC_xa
```

2. Set the VCS_HOME environment variable to the VCS installation directory and set the path variable to the VCS installation bin directory. The remaining steps are written for the C-shell.

```
% setenv VCS_HOME /vcs_installation_directory
% set path = ($VCS_HOME/bin $path)
```

3. For Synopsys Licenses, set the SNPSLMD_LICENSE_FILE environment variable to the license host and port, or the license file.

```
% setenv SNPSLMD_LICENSE_FILE port@host
```

```
- or -  
% setenv SNPSLMD_LICENSE_FILE license_file_name  
- for example -  
% setenv SNPSLMD_LICENSE_FILE \  
128@synopsys:7129@synopsys:$LM_LICENSE_FILE
```

4. Set the VERDI_HOME environment variable and include the path to Verdi in the UNIX path. The Verdi tool is needed to write out a digital FSDB file.

```
% setenv VERDI_HOME /verdi_installation_directory  
% set path = ($VERDI_HOME/bin $path)
```

5. Set the path to the Custom WaveView executable. The Custom WaveView tool is used to view the waveforms created during the mixed-signal simulation.

```
% set path = (waveview_installation_directory/bin $path)
```

Running an Analog-only (CustomSim) Simulation

The runXa script contains commands to compile and run the SPICE netlist and vectors in an analog-only simulation. The top.spi SPICE netlist contains the entire design and the xa.cfg file contains the CustomSim configuration settings. The adder.vec file contains the stimulus vector file for the simulation.

The xa.cfg file contains the following information:

```
set_sim_level -level 3  
  
# Command to probe voltage & current signals  
probe_waveform_voltage * -port 1 -level 99  
probe_waveform_current * -port 1 -level 99  
  
# Command to load the vector file  
load_vector_file -file adder.vec
```

Run the analog-only simulation with the runXa script as follows:

```
% runXa
```

After the simulation is finished, open the Custom WaveView tool and load the results_xa/xa.fsdb output file to view the waveforms as described in the next section.

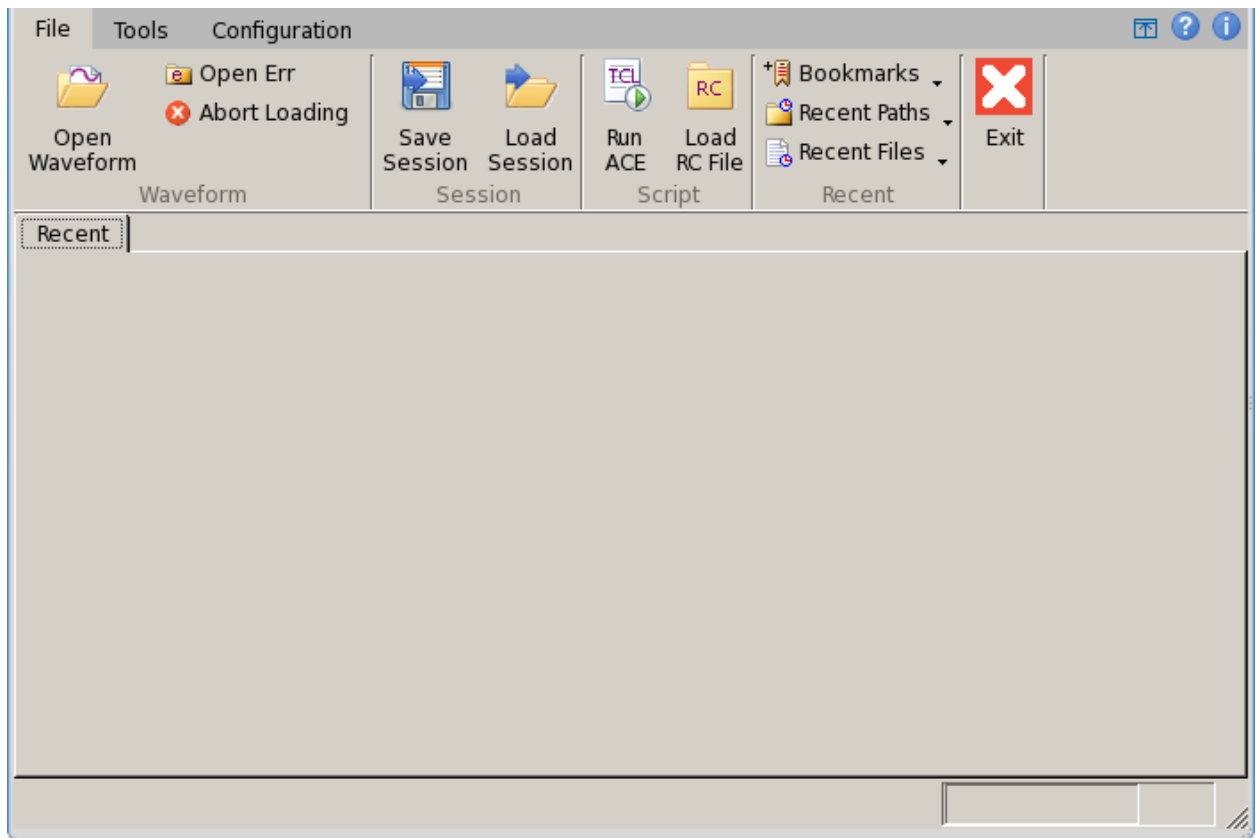
Viewing Waveforms in Custom WaveView

1. Run the following command from the UNIX command line to open the Custom WaveView GUI:

```
% wv &
```

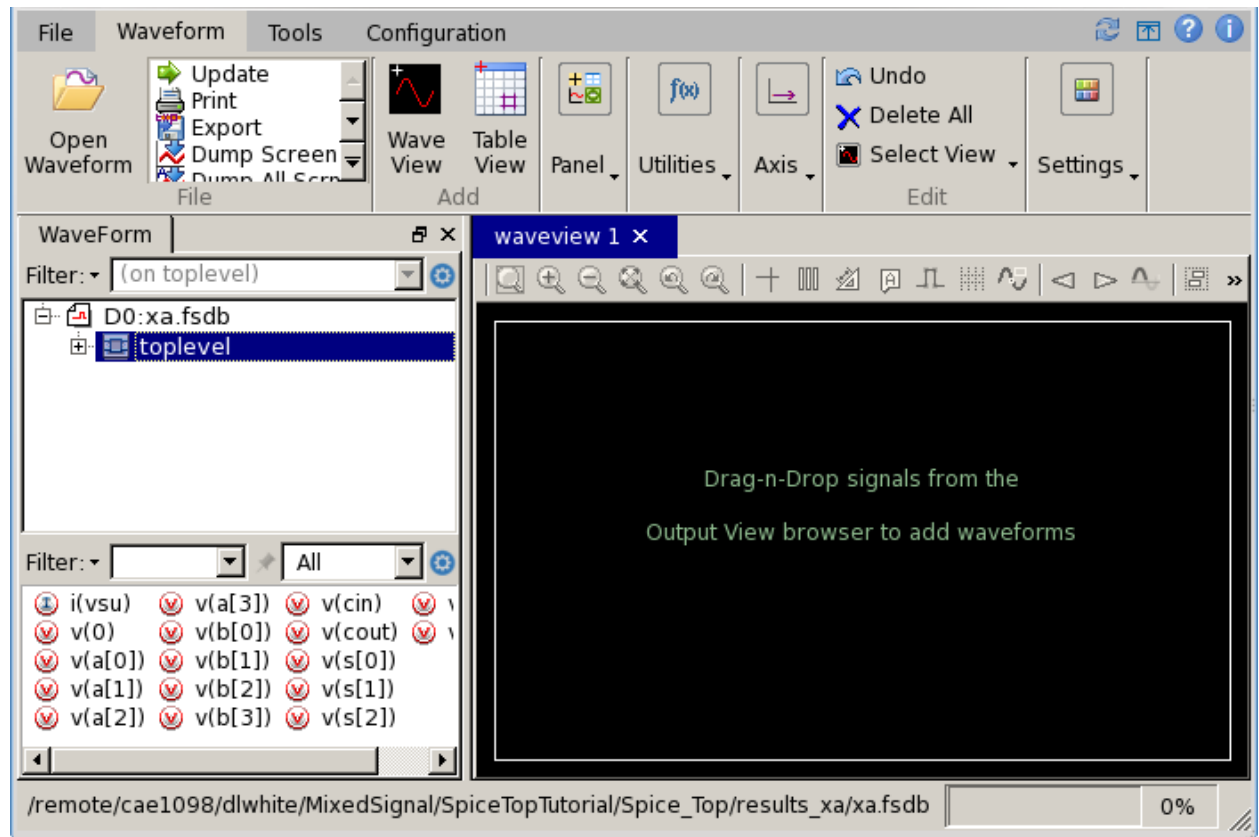
The `wv` command starts the Custom WaveView tool as shown in the following screenshot:

Figure 2 Custom WaveView



2. Click **Open Waveform** and load the `results_xa/xa.fsdb` file.
3. Double-click the "`D0:xa.fsdb`" label in the top left of the **WaveForm** tab to view the hierarchy. Next, double-click the "toplevel" label to display the list of signals.

Figure 3 View Hierarchy

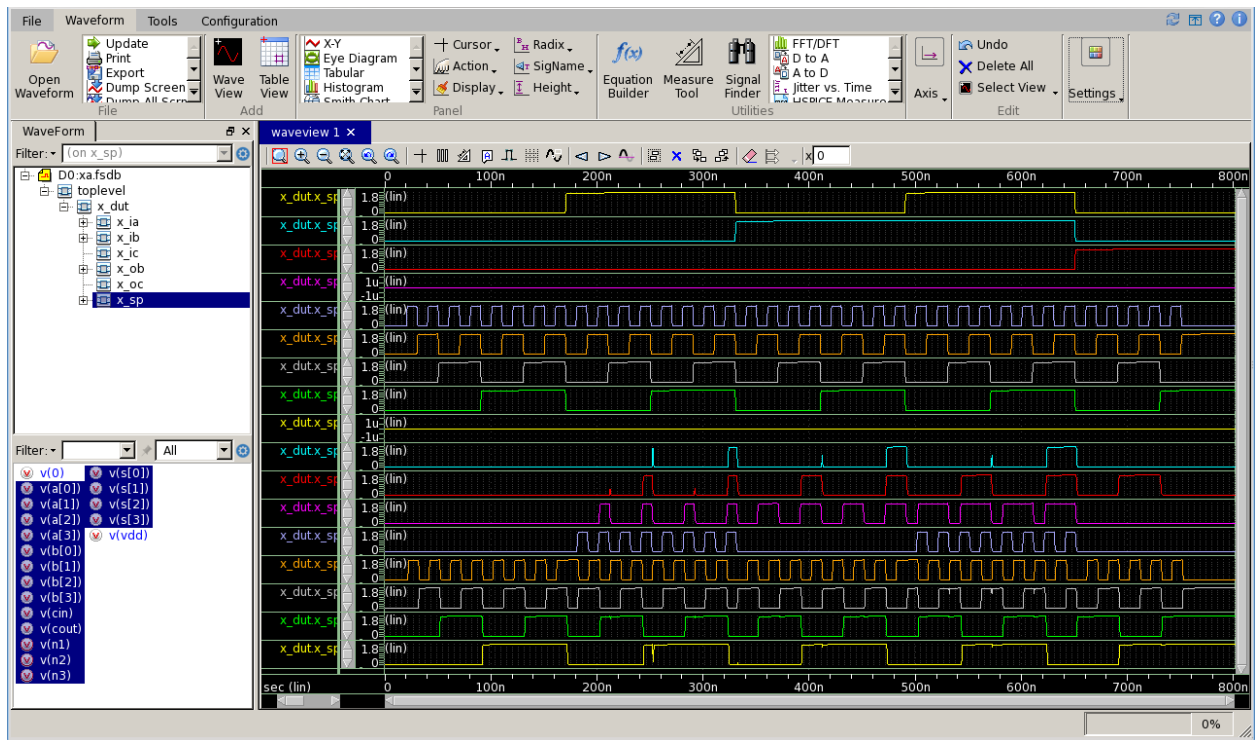


The Custom WaveView tool displays the simulation hierarchy in the top-left panel and signals for the selected hierarchy in the bottom-left panel.

4. Drag a module label from the hierarchy browser panel (top-left) of the **WaveForm** tab into the output view to display all input and output signals for the module. Alternatively, click a hierarchy label in the hierarchy browser and drag or double-click signal labels from the signal list panel (bottom left) into the waveform view to display a signal waveform.

The following screenshot shows the waveforms in the “top” module.

Figure 4 *x_dut/x_sp Module Waveforms*



Verify the adder results by checking that the s[3:0] and cout waveforms display the correct results based on the a[3:0], b[3:0], and cin input signals.

Running a Two-Step Mixed-Signal VCS AMS Simulation

The mixed-signal VCS AMS simulation can be run in a two-step flow or a three-step flow. The two-step flow used in the runVcsAms script includes both the compilation and simulation steps.

The following lines in the runVcsAms script compile the design:

```
# Compile
echo "verilog compilation"
vcs \
  -full64 \
  addr4.v \
  fsdbDump.v \
  -debug_access+all \
  -ad=vcsAD.init \
  -l ./results_vcsAms/vcs.log
```

In this script, the vcs command is used to parse the addr4.v and fsdbDump.v Verilog simulation files and enable mixed-signal simulation. The fsdbDump.v file has a Verilog module to write out the digital signals in FSDb format during simulation. The following options are used:

- The `-l` option specifies the name of the compile log file (`./results_vcsAms/vcs.log`).
- The `-debug_access+all` option enables debugging visibility and control, line-by-line simulation stepping, support for breakpoints, and other features.
- The `-ad` option specifies a mixed-signal simulation control file (`vcsAD.init`) and directs the `vcs` command to include both Verilog files and SPICE files when creating the simulation executable.
- The `-full64` option enables compilation and simulation in 64-bit mode.

The `vcsAD.init` file contains the following mixed-signal commands:

```
spice_top;
bus_format [%d];
use_verilog -module addr4;
choose xa top.spi -c xa.cfg -o results_vcsAms/xa/xa;
```

These mixed-signal commands are described below:

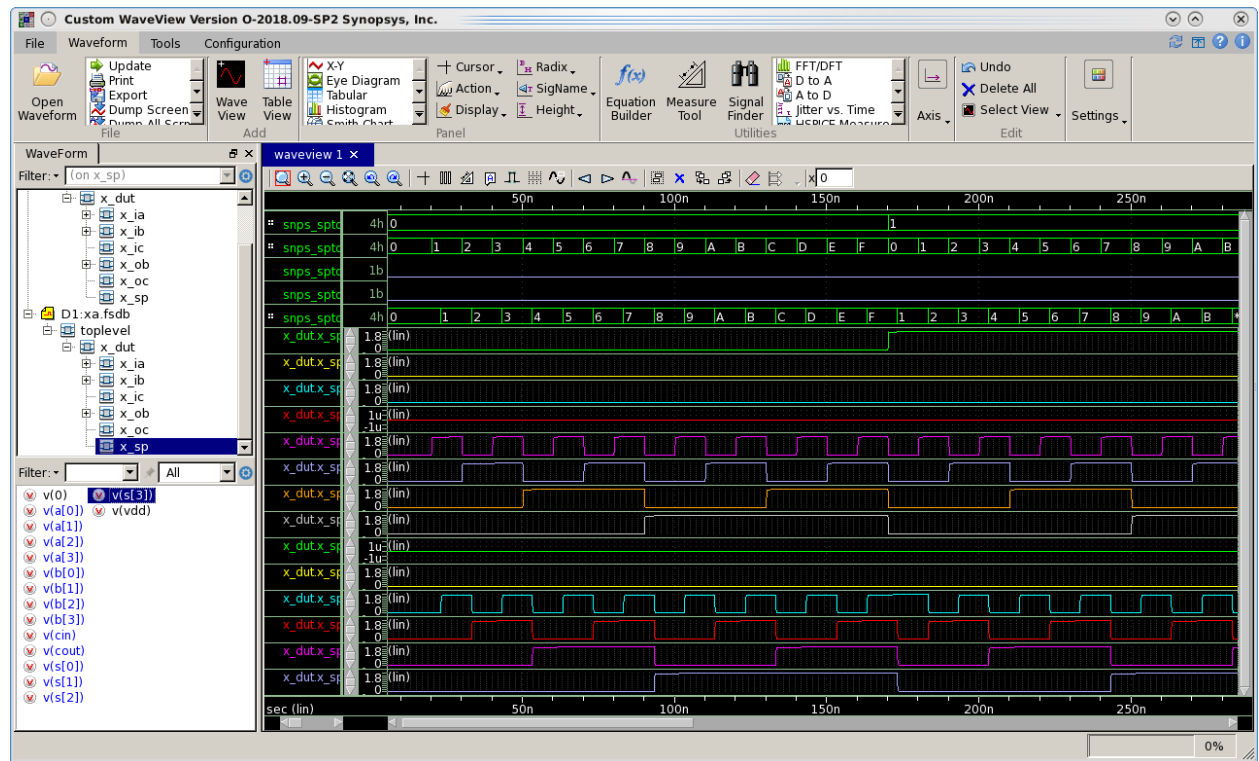
- `spice_top`: Specifies that the top-level design is described by using a SPICE netlist.
- `bus_format`: Specifies the character format for buses in the SPICE netlist. In this example, `bus_format [%d]` specifies that SPICE ports such as `a[3]`, `a[2]`, `a[1]`, and `a[0]` can be represented as a bus named “`a[3:0]`”. This allows a bus port named “`a`” in the Verilog module to map to SPICE ports `a[3]`, `a[2]`, `a[1]`, and `a[0]` in the SPICE netlist.
- `use_verilog`: Directs the tool to use a Verilog view for the specified module. In this example, `use_verilog -module addr4` forces all instances of `addr4` in the design to use the Verilog view of the module.
- `choose`: Specifies the analog simulation engine and command-line options for the analog simulation. In this example, the `xa CustomSim` simulation engine is specified and the `top.spi` SPICE file and `xa.cfg CustomSim` configuration file are used by CustomSim.

To run the mixed-signal simulation in the two-step flow, use the `runVcsAms` script.

```
% runVcsAms
```

After the simulation is finished, start the Custom WaveView tool by using the `wv` command, load the `results_vcsAms/vcs.fsdb` digital output file from the VCS tool, and the `results_vcsAms/xa/xa.fsdb` analog output file from the CustomSim tool to view the waveforms as discussed in the previous section.

Figure 5 Mixed-Signal Waveforms



Verify the adder outputs for given inputs and look at the signals at the analog-digital boundary.

Running a Three-Step Mixed-Signal VCS AMS Simulation

The three-step flow performs analysis, elaboration, and simulation as separate steps. The `runVcsAms_3step` script contains the commands to analyze, elaborate, and run the mixed-signal design. During design analysis, the tool verifies the syntax of Verilog files and generates intermediate files for design elaboration. Any syntax errors in Verilog netlists are reported during this step.

The following lines in the `runVcsAms_3step` script analyze the design:

```
echo "verilog analysis - vlogan"
vlogan \
  -full164 \
  addr4.v \
  fsdbDump.v \
  -l ./results_vcsAms_3step/vlogan.log
```

In this example, the `vlogan` command analyzes the Verilog files.

By default, intermediate files are stored in the current directory. You can override the default by defining logical libraries and corresponding physical libraries in the

synopsys_sim.setup file. In this example, the synopsys_sim.setup file contains the following:

```
WORK > default
default : ./WORKLIB
```

WORK is the default logical library and is mapped to the physical library named ./WORKLIB. In this example, the VCS MX tool stores all intermediate files in the ./WORKLIB library.

During design elaboration, the design hierarchy is constructed based on the information obtained during the design analysis step. At this stage, incorrect port connectivity or missing definitions for instantiated blocks in Verilog or SPICE are identified and reported.

The following lines in the script elaborate the design:

```
echo "design elaboration - vcs"
vcs \
  -full64 \
  -debug_access+all \
  fsdbDump \
  -ad=vcsAD_3step.init \
  -l ./results_vcsAms_3step/vcs.log \
  -o simv_3step
```

The specifications and commands in the vcsAD_3step.init mixed-signal control file are identical to those used in the two-step flow, except for the output directory specified by the -o option.

```
spice_top;
bus_format [%d];
use_verilog -module addr4;
choose xa top.spi -c xa.cfg -o results_vcsAms_3step/xa/xa;
```

To run the mixed-signal simulation by using a three-step flow, run the runVcsAms_3step script:

```
% runVcsAms_3step
```

Both two-step and three-step flows generate identical outputs. The waveforms generated in three-step flow can be viewed in a similar manner to two-step flow. Load the results_vcsAms_3step/vcs.fsdb and results_vcsAms_3step/xa/xa.fsdb files to view the digital and analog waveforms, respectively.

Understanding Mixed-Signal Simulation Report Files

In addition to the waveform output, mixed-signal simulation generates a set of report files and writes the reports to a directory with a .msv extension. In this example, the simv.msv directory contains the reports from the two-step simulation and the simv_3step.msv directory contains the reports from the three-

step simulation. Use the report files to confirm and debug the mixed-signal configuration. The files can be used to understand how the tool interprets and connects the mixed-signal interfaces in the design as described below.

hierarchy.rpt

The hierarchy.rpt file lists the hierarchical paths to all cells in the design together with their cell names as shown in the following example. Note that the simulation uses the digital view of the addr cell and the analog view for all other cells.

```
# () is used to denote digital_cell, e.g. (digital_ff)
# <> is used to denote analog_cell, e.g. <analog_ff>
# {} is used to denote ams_cell, e.g. {ams_ff}
# [] is used to denote verilog-A, e.g. [verilog-A]
x_dut<chip>.x_ia<bufr4>.x0<bufr>
...
x_dut<chip>.x_ib<bufr4>.x0<bufr>
...
x_dut<chip>.x_ic<bufr>
x_dut<chip>.x_ob<bufr4>.x0<bufr>
...
x_dut<chip>.x_oc<bufr>
x_dut<chip>.x_sp(addr4)
```

interface_element.rpt

The interface_element.rpt file contains information about the interface elements used at the mixed-signal boundary. The file also provides a list of resistance map files used in the design. The following is a section from the simv.msv/interface_element.rpt file.

```
Total number of resistors added by interface elements = 5
Total number of capacitors added by interface elements = 5

rmap_file 1 = /global/apps/xa_2018.09-SP3/include/rmapAD.init

a2d loth=0.9v hith=0.9v node=x_dut.ai[0];
a2d loth=0.9v hith=0.9v node=x_dut.ai[1];
a2d loth=0.9v hith=0.9v node=x_dut.ai[2];
a2d loth=0.9v hith=0.9v node=x_dut.ai[3];
a2d loth=0.9v hith=0.9v node=x_dut.bi[0];
a2d loth=0.9v hith=0.9v node=x_dut.bi[1];
a2d loth=0.9v hith=0.9v node=x_dut.bi[2];
a2d loth=0.9v hith=0.9v node=x_dut.bi[3];
a2d loth=0.9v hith=0.9v node=x_dut.cini;
d2a hiv=1.8v lov=0v node=x_dut.couti;
d2a hiv=1.8v lov=0v node=x_dut.si[0];
d2a hiv=1.8v lov=0v node=x_dut.si[1];
d2a hiv=1.8v lov=0v node=x_dut.si[2];
d2a hiv=1.8v lov=0v node=x_dut.si[3];
```

In this SPICE-top mixed-signal simulation, the circuit stimulus is provided in a vector file. The vectors drive analog buffers that connect to inputs on the digital addr4 block by using a2d interface elements. The digital outputs of the addr4 module are connected to the analog output buffers by using d2a interface elements. From the report, the threshold values for a2d conversion are set at the default: 50% of the VDD voltage. The high and low voltage values produced by d2a conversion are set to VDD and GND respectively.

mview.rpt

The mview.rpt file lists all cells in the design that have more than one view as shown in the following example:

```
; Has unresolved modules?
0
; For SPICE: case 1 by default
; Lists of modules: Verilog SPICE Verilog-A Adfmi
addr4 addr4 * *
; Verilog Top Module(s)
1 snps_sptop
;done
```

Note that the addr4 module has both Verilog and SPICE views available. Although is a SPICE-top mixed-signal design, the tool also creates a dummy Verilog top module named snps_sptop.

port.rpt

The port.rpt file contains information about how ports are mapped when both a SPICE view and an HDL view are available for use by the simulator. The following is a section of the simv.msv/port.rpt file.

```
----- Multiple cell views -----

----- Cell: addr4 -----
subckt addr4: file "top.spi" line "47" ports=14
verilog addr4: file "addr4.v" line "3" ports=14

verilog module addr4([3:0]a, [3:0]b, cin, [3:0]s, cout);
    input    a, b, cin;
    output   s, cout;
subckt addr4 a[3], a[2], a[1], a[0], b[3], b[2], b[1], b[0], cin,
    s[3], s[2], s[1], s[0], cout

use_verilog -cell addr4 port_map(
    *=>snps_by_name);

port_dir -cell addr4(
    input    a, b, cin;
    output   s, cout
); //derived from verilog
```

The port_map entry in the use_verilog command contains *=>snps_by_name; this indicates that all SPICE ports without a specific mapping assignment are mapped to Verilog ports by their port name. If some ports are mapped using a port_map command in the mixed-signal control file, those are also listed together with the default mappings.

use_cell_view.rpt

This file lists all the cells that match each of the use_spice, use_verilog, and use_vhdl commands. The following is a section of the simv.msv/use_cell_view.rpt file.

```
#=====
# Command used on line 2 of mixed-signal control file
# vcsAD.init followed by instances partitioned by that
# command
#=====
use_verilog -module addr4;

#x_dut.x_sp
```

Summary

This tutorial provides a basic overview of mixed-signal simulation and uses scripts and an example design to perform all-analog and mixed-signal simulations on a SPICE-top design:

- Analog-only simulation
- Mixed-signal VCS AMS two-step flow
- Mixed-signal VCS AMS three-step flow

In addition, the tutorial describes using the Custom WaveView waveform viewer to view the simulation results and lists the available report files generated during simulation.

Refer to the *Mixed-Signal Simulation User Guide* and *CustomSim User Guide* for more information about mixed-signal and CustomSim simulation flows and commands.